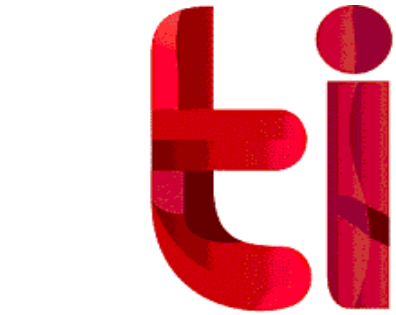
EVA: Smart College Event & Venue Management Platform

A Project Proposal submitted

For Software Engineering (UCS 503P) by

Palak	1024030631
Yashita Aggarwal	1024030644
Vatsal Kumar Singh	1024030626

Submitted to Ms Kirtika Goyal



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

SESSION-YEAR: JULY-DEC,2026

1 Higher-order goal

EVA aims to transform college event management from a fragmented, manually coordinated process into a unified and measurable workflow. The platform connects event creation, venue booking, student registration, QR-based footfall, feedback, and analytics so that club coordinators can manage an event from planning through post-event evaluation in one system.

The long-term goal is to enable better campus resource utilization and data-driven event planning, while keeping the initial implementation focused enough to be feasible as a Software Engineering course project.

2 Time-to-value

- We will deliver meaningful increments quickly by building the Core Platform first and layering modules incrementally: Booking → Registration → Footfall → Feedback & Analytics → AI Recommendation.
- We will prioritize fast feedback over perfection by validating the core booking workflow with users before adding the AI recommendation layer, keeping release friction low.
- Success is defined by what can be delivered and measured in the first phase (Core Platform + Event & Venue Booking), then improved based on coordinator and student feedback.

3 Problem Statement

College clubs often use multiple disconnected tools to organize an event. Venue availability may be checked separately, registrations may be collected through forms, communication may happen across different channels, footfall may be recorded independently, and feedback and analysis may be performed manually.

This fragmentation creates several practical problems:

- Manual venue booking and the possibility of scheduling conflicts.
- Scattered registration information and additional coordination effort.
- A gap between the number of registrations and actual event footfall.
- Feedback and event-performance information remaining disconnected.
- Limited consolidated data for improving future events.

Why this matters now (contextual relevance): The central problem addressed by EVA is the lack of a single, college-focused platform that connects the complete event lifecycle for clubs operating with limited coordination bandwidth.

4 Proposed Solution

4.1 Overview

EVA provides two primary user roles: Students and Club Coordinators. Students discover events, register, check in through QR, and provide feedback. Club Coordinators create and manage events, book venues, monitor registrations and footfall, and review event analytics. The platform is organized

around five core modules: Event & Venue Booking, Student Registration, QR-based Footfall, Feedback, and Analytics.

4.2 Core workflow

The proposed event lifecycle is:

Create Event → Book Venue → Student Registration → Event → QR Footfall → Feedback → Analytics

At the user-workflow level:

- **Student:** Discover → Register → QR Check-in → Feedback
- **Club Coordinator:** Create → Book → Manage → Analyze

Event & Venue Booking: This module captures event requirements — expected participant count, required date, start and end time, and required facilities — and matches them against venue capacity and availability. The workflow is:

Event Requirements → Venue Filtering → Availability Check → Conflict Detection → Suitable Venues → AI Recommendation → Final Booking

If two club coordinators attempt to book the same venue for overlapping time periods, the system must ensure that only one non-conflicting booking is confirmed; the other request is rejected or asked to choose an alternative venue or slot. This is enforced through transaction-safe, atomic booking validation at the database level so that concurrent booking attempts cannot both succeed for the same venue and time window.

QR-based Footfall: At check-in, the QR scan first verifies that the student is registered for that event, then records actual participation, and prevents duplicate check-ins for the same registration. This allows the system to directly compare Registered Students vs. Actual Footfall, closing the gap identified in the problem statement.

4.3 Operational constraints for an educational, campus setting

- Keep the scope intentionally limited to two user roles and five core modules so the MVP stays manageable.
- Use commonly available development environments, database systems, QR functionality, and software libraries rather than specialized infrastructure.
- Design modules so they can be tested independently and integrated incrementally.

This scope keeps the project feasible within a semester: only two roles and five modules need to be implemented, development proceeds incrementally, the AI layer is added only after the core deterministic booking workflow is working, and no specialized hardware is required beyond a standard web/database deployment targeted at a single college.

5 Solution Approach

This is an engineering course project, so the focus is on connected workflow, automation, and system correctness rather than novel algorithm research.

5.1 Smart venue recommendation

AI operates only after deterministic filtering has already narrowed the venue set. The backend business rules — not the AI layer — remain the source of truth for availability, capacity, and conflicts. The recommendation pipeline is:

Coordinator Input → Event Requirements → Deterministic Venue Filtering → Availability Check → Conflict Detection → Valid Venues → AI Ranking → Recommended Venue → Coordinator Confirmation → Final Booking

- Rank suitable, available venues using event requirements such as capacity fit, timing, required facilities, and historical venue utilization.
- Introduce the recommendation layer only after the core deterministic booking workflow (availability check and conflict detection) is functional, without claiming state-of-the-art AI or guaranteed accuracy.
- Present ranked venue suggestions to the coordinator for final confirmation; the system never auto-books without coordinator approval.

At the beginning, historical event data may be limited, so recommendation quality is expected to improve gradually as EVA accumulates more event and venue usage data; this is treated as an accepted, manageable limitation rather than a blocking risk.

5.2 Web architecture

A typical 3-tier web architecture underlies the five core modules:

- **Frontend:** responsive UI for students and coordinators (event discovery, registration, QR check-in, dashboards).
- **Backend API:** authentication, event/venue booking logic, registration, footfall, and feedback handling.
- **Database:** stores users, events, venues, bookings, registrations, footfall records, and feedback.

Conceptually, the modules are not independent silos but form one connected data chain across the event lifecycle:

- Club Coordinator → Event → Venue Booking
- Student → Event Registration → QR Footfall
- Event → Feedback → Analytics

Each event record links a coordinator, a confirmed venue booking, a set of student registrations, footfall entries, and feedback responses, which is what allows the analytics module to report meaningfully across the whole lifecycle rather than on isolated tables.

5.3 Incremental build and fast delivery enabler

- Build and test each module independently before integrating it into the platform.
- Integrate modules incrementally following the phased plan: Core Platform → Booking → Registration → Footfall → Feedback & Analytics → AI Recommendation.

- Keep each phase deployable on its own so the team always has a working, demonstrable system.

This incremental plan follows the standard Software Engineering lifecycle end to end: Requirements — identify student and club coordinator needs and define functional/non-functional requirements; Design — define workflows, system architecture, database structure, interfaces, and module interactions; Implementation — develop the core platform and modules incrementally; Testing — perform module-level, integration, API, booking-conflict, QR, and workflow testing; Deployment — deploy the integrated MVP; Maintenance — fix defects, improve performance, refine recommendations, and extend the system.

6 Evaluation Criterion

We define success using measurable metrics tied to the core event-and-venue-booking workflow.

6.1 Primary evaluation metric

Time-to-Venue-Confirmation (TTVC): the time between an event/venue booking request and the confirmation of a valid, conflict-free booking status in the system.

- **Target:** TTVC will be measured during pilot testing rather than fixed in advance, since a realistic target depends on observed usage patterns and current manual booking baselines.
- **Attribution:** measured from database timestamps (booking request vs. confirmed-status update).

6.2 Secondary evaluation metrics

- **Conflict-detection accuracy:** percentage of overlapping bookings correctly flagged before confirmation.
- **Registration completion rate:** percentage of started registrations that complete successfully within capacity limits.
- **Footfall capture reliability:** percentage of registered attendees whose QR check-in is correctly recorded, including successful prevention of duplicate check-ins.
- **Feedback response rate:** percentage of attendees submitting structured feedback after an event.
- **Recommendation relevance:** coordinator-rated usefulness (1–5) of AI-suggested venues.

6.3 Pilot validation plan

- Recruit 2–3 club coordinators and 15–20 student participants for a pilot event cycle.
- Compare metrics before/after within the iteration window, using coordinator interviews to estimate baseline (pre-EVA) booking and coordination time.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **MVP scope**: a single college, a limited number of clubs and venues, and a controlled event volume, which keeps the pilot manageable and measurable.
2. **Future scope**: more clubs, more venues, more events, and more students, extending eventually to multiple campuses or institutions while preserving the same role-based workflow.
3. **Realistic scaling directions** include a modular backend, database indexing on frequently queried fields (venue, date, event), stateless APIs, caching for read-heavy views such as event listings, and horizontal backend scaling if load increases.
4. **Registration integrity**: high registration concurrency must not cause over-registration beyond venue capacity; this is enforced by the same atomic, transaction-safe validation used for booking conflicts.

Operationally, individual modules can be tested independently and integrated incrementally, and the system can later add administrative roles, richer reporting, institutional-calendar integrations, notifications, and more advanced recommendation models as scope expands.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement EVA as a web-based project:

- Web framework for REST APIs and responsive pages.
- Managed relational or document database for users, events, venues, and bookings.
- Authentication approach (token-based or session-based) for Students and Club Coordinators.
- QR generation/scanning library for footfall check-in.
- Test framework for unit/integration tests of each module.
- CI runner and staging deployment target for incremental releases.

These mature components let the team focus on workflow design, correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

The MVP includes authentication, both user roles (Student and Club Coordinator), event creation, venue data, availability checking, capacity validation, conflict detection, venue booking, student registration, QR footfall, feedback, analytics, and the AI Smart Venue Recommendation layer. The core deterministic booking workflow is built first; the AI recommendation layer is added on top of it, not as a replacement for it. No waitlist functionality is included.

9.1 Initial deliverable

- Finalized requirements and user flows for Students and Club Coordinators.
- Core platform and database design.
- Event and Venue Booking module with availability check and conflict detection.

9.2 Subsequent deliverables

- Student Registration and QR-based footfall workflow.
- Feedback collection and post-event analytics dashboard.
- AI-powered smart venue recommendation layer (MVP).
- Testing, refinement, and integrated deployment.

10 Risks and mitigations

- **Scope creep**: keep the MVP limited to two roles and five core modules.
- **Booking logic complexity**: implement deterministic availability and conflict rules before adding AI recommendation.
- **Concurrent/double booking**: use transaction-safe, atomic booking validation so overlapping requests for the same venue cannot both be confirmed.
- **AI recommendation quality**: use constrained, explainable recommendations based on available venue data, reviewed by the coordinator before final booking; expect quality to improve as data accumulates.
- **QR misuse / duplicate check-in**: validate registration status at scan time and reject repeat check-ins for the same registration.
- **Data accuracy**: validate registrations, QR check-ins, and feedback inputs.
- **Integration issues**: develop and test modules independently before full integration.
- **Semester time constraints**: sequence modules by priority (Core Platform → Booking → Registration → Footfall → Feedback & Analytics → AI) so a working, demonstrable system exists at every phase even if later modules slip.

11 Summary

This project proposes EVA, a deployable web platform that unifies event planning, venue booking, student registration, QR-based footfall, feedback, and analytics for college clubs. It meets the course criteria by emphasizing time-to-value through incremental, phase-wise delivery, keeping the core booking workflow deterministic before adding AI, and using measurable evaluation criteria (especially time-to-venue-confirmation). The project remains deliberately scoped around two user roles and five core modules, making it practical for incremental development while remaining extensible for future campuses and richer analytics.